

ADC: Jembatan Dunia Analog ke Digital

Praktikum Embedded Systems — Modul 04: Analog-to-Digital Conversion

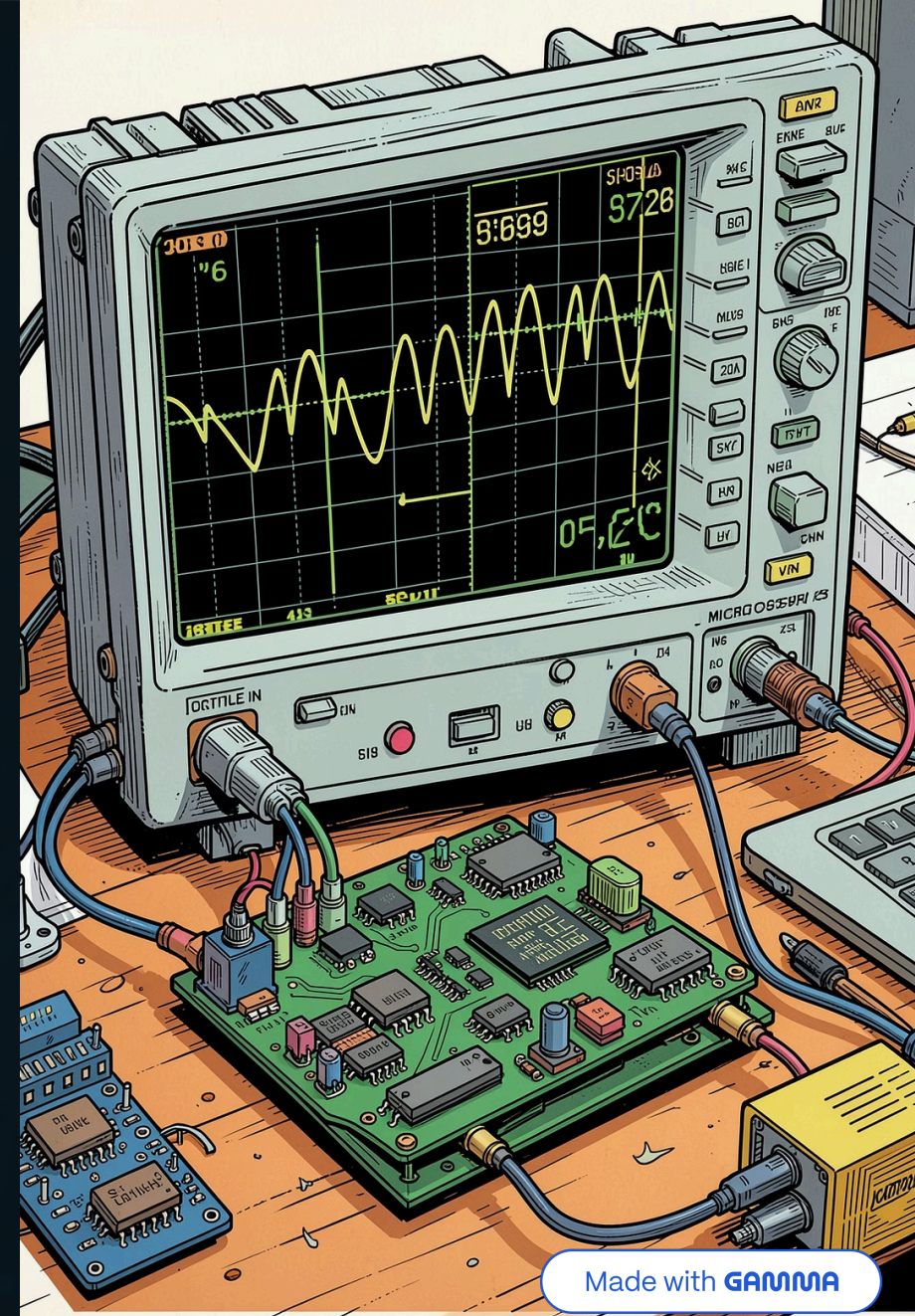
ADC (Analog-to-Digital Converter) adalah komponen fundamental dalam setiap sistem embedded modern. Tanpa ADC, mikrokontroler tidak dapat "merasakan" dunia nyata — tidak dapat mengukur suhu, cahaya, tegangan baterai, atau sinyal dari sensor apapun. Modul ini membahas teori ADC secara mendalam, teknik filtering sinyal, implementasi pada ESP32 dan STM32, serta 12 eksperimen praktikum yang membangun kompetensi nyata dalam akuisisi data analog.

EMBEDDED SYSTEMS

ADC THEORY

ESP32 & STM32

12 EKSPERIMEN



Made with GAMMA

Teori ADC: Tahapan Konversi dan Teorema Nyquist

Proses konversi ADC berlangsung dalam tiga tahap berurutan yang terjadi di dalam hardware ADC hanya dalam hitungan microsecond. Setiap tahap memiliki peran kritis dalam memastikan representasi digital yang akurat dari sinyal analog kontinu.

Tiga Tahap Konversi ADC

1 Sampling

Mengambil nilai amplitudo sinyal analog pada interval waktu tertentu (periode sampling $T_s = 1/f_s$). Semakin tinggi frekuensi sampling, semakin akurat representasi sinyal asli.

2 Kuantisasi

Memetakan nilai kontinu ke level diskret sesuai resolusi bit. ADC 12-bit memiliki 4096 level kuantisasi (0–4095). Semakin banyak bit, semakin kecil jarak antar level dan semakin presisi hasilnya.

3 Encoding

Mengubah level kuantisasi menjadi kode biner murni yang dapat diproses CPU. Output akhir adalah angka digital yang merepresentasikan tegangan input.

Teorema Nyquist-Shannon

Teorema fundamental yang menentukan frekuensi sampling minimal untuk merekonstruksi sinyal tanpa distorsi:

❗ $f_s \geq 2 \times f_{\max}$ — Frekuensi sampling harus minimal dua kali frekuensi tertinggi dalam sinyal input.

Jika teorema ini dilanggar, terjadi **aliasing** — sinyal frekuensi tinggi "menyamar" sebagai sinyal frekuensi rendah, menghasilkan pembacaan yang salah dan tidak dapat diperbaiki secara software.

- Suara manusia (20 kHz) → sampling minimal 40 kHz (standar audio CD: 44.1 kHz)
- Sensor AC 50 Hz → sampling minimal 100 Hz (praktis: 500–1000 Hz)
- Sinyal sensor variabel cepat → hitung $f_{\max}$ terlebih dahulu sebelum menentukan f_s

Aliasing dapat dicegah dengan **anti-aliasing filter** (low-pass filter analog) sebelum input ADC, yang membatasi bandwidth sinyal di bawah $f_s/2$.

Resolusi, Kualitas Sinyal, dan Parameter ADC

Resolusi dan Formula Konversi

Resolusi ADC menentukan jumlah level diskret yang dapat dibedakan. Semakin tinggi resolusi, semakin kecil perubahan tegangan minimum yang dapat dideteksi (LSB size).

10-bit

$2^{10} = 1.024$ level
LSB ≈ 3.22 mV @ 3.3V

12-bit

$2^{12} = 4.096$ level
LSB ≈ 0.805 mV @ 3.3V

16-bit

$2^{16} = 65.536$ level
LSB ≈ 0.050 mV @ 3.3V

❏ **Rumus Konversi ($V_{ref} = 3.3V$):**
 $V_{mV} = (raw \times 3300) \div 4095$
Contoh: raw = 2048 $\rightarrow V = 1650$ mV

Setiap bit ekstra menggandakan jumlah level dan mengurangi LSB size setengahnya — tetapi CPU lebih sibuk dan konsumsi daya meningkat. Trade-off desain harus disesuaikan dengan kebutuhan aplikasi.

Parameter Kualitas Sinyal ADC

SNR (Signal-to-Noise Ratio)

ADC 12-bit ideal: SNR ≈ 74 dB. Rumus teoritis:
 $SNR = 6.02 \times N + 1.76$ dB.
SNR lebih tinggi = sinyal lebih bersih dari noise kuantisasi.

Quantization Error

Error maksimum = $\pm \frac{1}{2}$ LSB. Untuk ADC 12-bit @ 3.3V: $\approx \pm 0.4$ mV. Error ini tidak dapat dihilangkan, hanya dikurangi dengan menambah resolusi.

INL (Integral Non-Linearity)

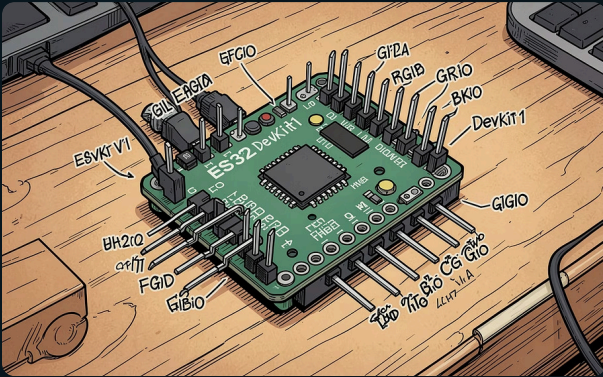
Deviasi maksimum kurva transfer ADC dari garis lurus ideal. ADC berkualitas industri: INL $< \pm 1$ LSB. INL besar menyebabkan error sistematis di seluruh range.

DNL (Differential Non-Linearity)

Variasi ukuran step antar level kuantisasi. DNL > 1 LSB dapat menyebabkan missing codes — nilai digital tertentu tidak pernah muncul.

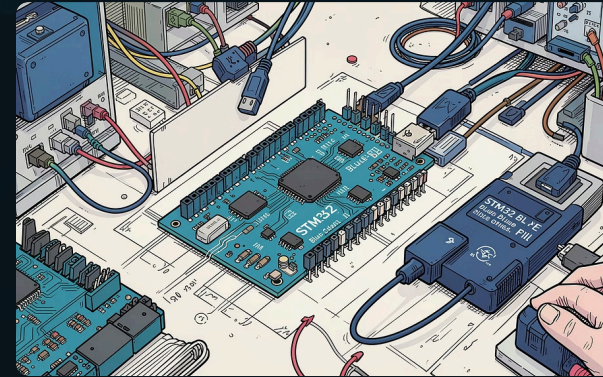
Fitur ADC: ESP32 vs STM32

Kedua platform memiliki arsitektur ADC yang berbeda secara fundamental. Memahami karakteristik masing-masing sangat penting untuk memilih platform yang tepat sesuai kebutuhan aplikasi.



ESP32 — Fleksibel untuk IoT

- **2 ADC unit**, total 18 channel analog
- **ADC1 (GPIO32–39):** Aman, tidak konflik WiFi — gunakan untuk sensor kritis
- **ADC2:** Berbagi modul WiFi — **tidak dapat digunakan** saat WiFi aktif transmit
- **Resolusi:** 9–12 bit selectable via API
- **Atenuasi:** 0 / 2.5 / 6 / 11 dB — memperluas range input 0–3.3V
- **Kalibrasi eFuse:** `adc_cal_raw_to_voltage()` — kompensasi non-linearitas chip otomatis
- **DMA:** `esp_adc_continuous_new_handle()` untuk akuisisi kecepatan tinggi



STM32 — Presisi untuk Industri

- **ADC SAR** (Successive Approximation Register): Stabil, linear, reliable untuk aplikasi industri
- **Resolusi**: 12-bit fixed — konsisten dan dapat diprediksi
- **Analog Watchdog**: Hardware otomatis deteksi threshold — zero CPU overhead, response instant
- **Channel Internal**: TEMPSENSOR (suhu chip), VREFINT (1.2V referensi stabil)
- **DMA**: Transfer data ke RAM tanpa intervensi CPU — ideal untuk sampling tinggi
- **Kalibrasi**: `HAL_ADCEX_Calibration_Start()` — sequence internal pabrik
- **Scan Mode**: Sampling simultan multi-channel tanpa lag sequential

⚠ Catatan Kritis ESP32: Jangan gunakan ADC2 saat WiFi aktif. Gunakan ADC1 (GPIO32–39) untuk semua sensor dalam aplikasi WiFi-connected.

Teknik Filtering Digital Sinyal ADC

Sinyal ADC mentah hampir selalu mengandung noise. Filtering digital adalah teknik wajib dalam setiap sistem pengukuran untuk mendapatkan data yang stabil dan akurat. Pilih teknik filtering berdasarkan karakteristik sinyal dan kebutuhan responsivitas aplikasi.



Moving Average Filter

Menghitung rata-rata N sampel terakhir menggunakan buffer circular yang diupdate kontinyu. Sangat efektif untuk sinyal statis atau berubah lambat.

- Rumus: $\text{filtered}[n] = (\sum \text{sampel terakhir } N) / N$
- SNR improvement: $\sqrt{N}$ kali ($N=10 \rightarrow 3.16\times$ perbaikan)
- Trade-off: lag kecil pada respons — N besar = lebih smooth tapi lebih lambat
- Rekomendasi: $N=10$ untuk umum, $N=5$ untuk responsif, $N=20$ untuk sangat smooth



Exponential Moving Average (EMA)

Memberikan bobot lebih tinggi pada sampel terbaru, sehingga lebih responsif terhadap perubahan sinyal dibanding moving average konvensional.

- Rumus: $\text{filtered}[n] = \alpha \times \text{raw}[n] + (1-\alpha) \times \text{filtered}[n-1]$
- α (alpha) = 0.1–0.3: responsif; $\alpha = 0.01$ –0.05: sangat smooth
- Tidak perlu buffer — hanya simpan 1 nilai sebelumnya (hemat RAM)
- Ideal untuk sinyal dinamis yang perlu tracking cepat



Median Filter

Mengambil nilai tengah (median) dari N sampel yang diurutkan. Sangat efektif menghilangkan spike noise acak dan outlier tanpa mempengaruhi sinyal utama.

- Urutkan N sampel, ambil nilai di posisi tengah
- N ganjil: 3, 5, atau 7 sampel ($N=5$ paling umum)
- Sangat efektif untuk noise impulsif (spike tajam sesaat)
- Kombinasikan dengan moving average untuk hasil optimal



Oversampling

Sampling pada frekuensi $N\times$ lebih tinggi, lalu rata-ratakan untuk meningkatkan resolusi efektif tanpa mengubah hardware ADC.

- $4\times$ oversampling $\rightarrow +1$ bit resolusi efektif
- $16\times$ oversampling $\rightarrow +2$ bit (12-bit $\rightarrow$ 14-bit efektif)
- $64\times$ oversampling $\rightarrow +3$ bit (12-bit $\rightarrow$ 15-bit efektif)
- Trade-off: waktu proses lebih lama, cocok untuk sinyal lambat/statis

Persiapan Hardware dan Eksperimen P01–P03

Komponen dan Rangkaian Wajib

- Potensiometer $10k\Omega \times 2$ (sensor utama)
- LDR (Light Dependent Resistor) $\times 1$
- Resistor $10k\Omega \times 3$ (voltage divider)
- LED merah, hijau, biru $\times 1$ masing-masing
- Multimeter digital (referensi akurasi)
- Breadboard + kabel jumper secukupnya
- ESP32 DevKit V1 / STM32 Blue Pill + ST-Link V2

⚠ Penting: Gunakan VCC 3.3V, jangan 5V — GPIO ADC hanya toleran 3.3V maksimum. Wiper potensio ke GPIO34 (ESP32) atau PA0 (STM32).

P01 — Single Read: Membaca Nilai Mentah ADC

Eksperimen dasar pertama: membaca nilai raw ADC dari potensio dan mengonversinya ke tegangan.

- ESP32: `adc_oneshot_read()` + `adc_cali_raw_to_voltage()`
- STM32: `HAL_ADC_Start()` + `HAL_ADC_GetValue()`
- Atenuasi 11 dB untuk range penuh 0–3.3V
- Serial Monitor 115200 baud, putar potensio 0–10k Ω
- Amati raw 0–4095, hitung $V_{mV} = (\text{raw} \times 3300) \div 4095$
- Contoh: raw 2048 $\rightarrow$ 1650 mV; bandingkan dengan multimeter

P02 — Voltage Display: Konversi Tegangan Serial

Tampilkan hasil ADC dalam millivolt (mV) di Serial Monitor. Putar potensio perlahan, amati tegangan naik-turun linear.

P03 — Moving Average Filter

Implementasi buffer circular $N=10$. Bandingkan raw (fluktuasi 100–110) vs filtered (stabil ~105). Hitung standar deviasi σ sebelum dan sesudah filter untuk kuantifikasi pengurangan noise.

Eksperimen P04–P07: Multi-Channel, Kalibrasi, DMA, dan Threshold

P04 — Multi-Channel ADC

Baca dua channel ADC simultan dari dua potensiometer berbeda (CH1: GPIO34/PA0, CH2: GPIO35/PA1). Tampilkan tabel 4 kolom: CH1_raw, CH1_mV, CH2_raw, CH2_mV yang update kontinyu.

Test Crosstalk: Putar pot1 ke minimum, pot2 ke maksimum — catat. Tukar posisi. Jika CH1 tidak berubah saat pot2 diputar = tidak ada crosstalk (bagus). STM32 Scan Mode lebih unggul karena sampling simultan; ESP32 polling sequential memiliki lag kecil yang acceptable.

P05 — ADC Calibration

Kalibrasi ADC untuk kompensasi non-linearitas pabrik dan meningkatkan akurasi pengukuran secara signifikan.

ESP32: `adc_cali_create_scheme_curve_fitting()` — load data eFuse otomatis (Two Point calibration paling presisi).

STM32: `HAL_ADCEX_Calibration_Start()` — sequence kalibrasi internal hardware.

Hasil: Error tanpa kalibrasi $\approx 5\%$ (terbesar di ujung range 0V dan 3.3V), dengan kalibrasi $\approx 1\%$ merata di seluruh range. Ambil 5 posisi potensiometer (0%, 25%, 50%, 75%, 100%) dan catat V_{ref} , V_{tanpa} , V_{dengan} untuk perbandingan.

P06 — Continuous DMA

Direct Memory Access (DMA) mentransfer data ADC ke RAM secara otomatis tanpa intervensi CPU — ideal untuk akuisisi data kecepatan tinggi.

- ESP32: `esp_adc_continuous_new_handle()`, `sample_freq_hz = 20.000 Hz`
- Buffer 256–512 sampel, callback saat buffer penuh
- P01 single-shot: ~ 100 SPS $\rightarrow$ P06 DMA: ~ 10.000 SPS (100× lebih cepat!)
- STM32 DMA: hingga ~ 100.000 SPS potensial
- Buffer terlalu kecil $\rightarrow$ overflow; terlalu besar $\rightarrow$ delay lama

P07 — Threshold Alert LED

Sistem peringatan ambang batas ADC dengan indikator LED warna-warni untuk status visual instant.

- LOW threshold = 1000 ($\sim 0.8V$) $\rightarrow$ LED biru menyala
- HIGH threshold = 3000 ($\sim 2.4V$) $\rightarrow$ LED merah menyala
- NORMAL (antara threshold) $\rightarrow$ semua LED off
- Putar potensiometer min–max, amati LED berubah otomatis
- **STM32 Watchdog:** Mode SINGLE_REG, set High/Low threshold, ITMode aktif — hardware monitoring otomatis, zero CPU overhead, response instant untuk safety-critical applications

Eksperimen P08–P12: Sensor Lanjut dan Analisis Statistik

P08 — Battery Monitor (Voltage Divider)

Monitor tegangan baterai Li-Ion menggunakan voltage divider resistor untuk menurunkan Vbat ke range aman ADC.

- Rangkaian: Vbat → [10kΩ] → ADC_pin → [10kΩ] → GND
- $V_{bat} = 2 \times V_{adc}$ (rasio pembagi 1:1)
- Li-Ion 4.2V → ADC 2.1V (aman untuk GPIO 3.3V)
- Mapping kapasitas: 4.2V=100%, 4.0V=80%, 3.7V=50%, 3.4V=20%, 3.0V=0%
- Sampling setiap 1 menit, log ke SD card untuk battery health tracking

P09 — Internal Temperature Sensor

Sensor suhu terintegrasi di dalam chip — gratis, tidak perlu komponen eksternal.

- ESP32: API built-in atau rumus manual
- STM32: Channel TEMPSENSOR, Vref 1.2V internal stabil
- Rumus STM32: $T = (V_s - 0.76) / (-2.5 \text{ mV}) + 25^\circ\text{C}$
- Normal idle: 25–30°C; saat CPU bekerja berat: naik terlihat
- Sample 239.5 cycle max untuk presisi tertinggi

P10 — Sampling Rate Measurement

Ukur kecepatan sampling aktual (SPS = Samples Per Second) yang dapat dicapai sistem.

- Code: start=micros(), baca N=1000, $\text{SPS} = 1.000.000 \times N / (\text{end} - \text{start})$
- ESP32 single-shot: ~100 SPS; DMA: ~10.000 SPS
- STM32 single-shot: ~10.000 SPS; DMA: ~100.000 SPS
- Bottleneck: ADC conversion time, UART baud rate (115200), loop delay

P11 — Light Sensor LDR

Voltage divider: 3.3V → [10kΩ] → ADC → [LDR] → GND. Terang = R kecil = tegangan ADC tinggi. Gelap = R besar = tegangan ADC rendah. Pilih R1=10kΩ (nilai tengah antara R_LDR terang 1kΩ dan gelap 100kΩ) untuk dynamic range optimal.

P12 — Statistical Analysis

Kumpulkan N=100 sampel, hitung: minimum, maksimum, mean, standar deviasi $\sigma = \sqrt{(\sum(x - \text{mean})^2 / N)}$. SNR = $20 \times \log_{10}(\text{mean} / \sigma)$ dB. Bandingkan σ raw vs filtered — σ lebih kecil = noise lebih rendah = SNR lebih tinggi = kualitas sinyal lebih baik.

Project: GreenAir Indonesia — Sistem Monitoring Udara Gedung

Studi kasus integrasi lengkap: sistem monitoring kualitas udara gedung perkantoran 3 lantai menggunakan dual-MCU architecture dengan ESP32 sebagai hub akuisisi data dan STM32 sebagai controller alert threshold.

Arsitektur Dual-MCU

- ESP32 Hub (Akuisisi Data)
- 4 channel ADC parallel: suhu (CH6/GPIO34), cahaya LDR (CH7/GPIO35), gas (CH4/GPIO32), baterai (CH5/GPIO33). Filter moving average + kalibrasi eFuse aktif. Mode normal: sampling 1 detik via DMA. Mode anomali: switch ke 10.000 SPS continuous. TX data ke STM32 via UART (GPIO2 TX → PA10 RX).
- STM32 Alert Controller
- Terima data via UART (PA9 TX → PA10 RX), parsing threshold logic: suhu >35°C atau gas >70% → trigger anomali. LED merah + buzzer aktif nonstop. Watchdog hardware monitoring otomatis. Logging timestamp setiap event anomali. Auto-recovery: kembali normal setelah 30 detik stabil.

Pin Mapping Lengkap

Sensor	ESP32 GPIO	STM32 Pin
Suhu	GPIO34 (ADC1_CH6)	PA0 (ADC1_IN0)
Cahaya LDR	GPIO35 (ADC1_CH7)	PA1 (ADC1_IN1)
Gas	GPIO32 (ADC1_CH4)	PA4 (ADC1_IN4)
Baterai	GPIO33 (ADC1_CH5)	PA9 (ADC1_IN9)
LED Alert	GPIO2	PB0
Buzzer	GPIO4	PB1
UART TX	GPIO1	PA9
UART RX	GPIO3	PA10

Rencana Eksekusi 2 Minggu

- Minggu 1:** P01–P07 dasar dual-platform, P04 multi-channel crosstalk test, P05–P06 kalibrasi + DMA verify, P07 threshold LED
- Minggu 2:** Dual-MCU UART komunikasi, mode switching normal/anomali, hardware assembly final, kode optimization dan tuning